

A20OS：高性能混合内核操作系统

从 MCU 到完整桌面：双重 ABI · 八大平台 · 工程化并发安全

AAAAAAAAAAAAAAAAAAAAA

武汉大学 | 2026 年 8 月



目录

项目概览

核心架构

并发与调度

内存管理

Linux ABI 兼容

Native ABI

混合内核服务与 IPC

文件系统与网络

图形与桌面

性能

驱动与平台

工程化与验证

未来工作

01

项目概览

19.2 万

自研代码行数

500

双 ABI syscall (361 +
139)

8

指令集架构

483

次提交 / 20 周连续演
进

A200S 是什么？——一页看完成度

- **混合内核**：性能路径留内核，易崩服务迁用户态
- **双重 ABI**：361 Linux syscall 兼容 + 139 Native syscall
- **八大平台**：7 hosted + ARMv7-M；星光 2 / LS2K1000 / STM32
- **完整桌面**：Wayland + LVGL 自研桌面 + virtio-gpu 3D
- **工程化**：锁契约文档化、54 项冒烟门禁、七架构矩阵

同一内核：从 64 KiB Flash 的 MCU，跑到 QEMU 桌面。

180/180

BuildStorm 用例全过（双架构）

54

项行为冒烟门禁（make smoke-*）

92 篇

设计文档，1.6 万行，随码同仓

从初赛到决赛：能力跃迁

维度	初赛	决赛（当前）
支持架构	4 个	7 hosted + ARMv7-M MCU
Linux ABI	部分 syscall	361 全表覆盖 ，含 io_uring / landlock / mseal
Native ABI	90 syscall	139 syscall + Pager / monitor 深化
调度器	8 级 MLFQ	EEVDF + per-CPU 队列 + 空闲窃取
桌面	-	Wayland 桌面 + virtio-gpu 3D
混合内核服务	-	svcmgr / 用户态驱动 / KEP 扩展
BuildStorm 耗时	2551 s / 2315 s (RV / LA)	1483 s / 1399 s (-42% / -40%)

从“能启动、能跑几个程序”进化到**可跑完整桌面、可跑决赛高性能负载**的通用内核。

从零开始：不基于任何现有内核

- **首个 commit** (2026-04-11): 8,052 行最小内核 + 基础用户程序; 此后 **483 次提交、20 周连续演进**
- **无一次性导入**: 最大单 commit 即首个最小内核, 其余全部是 $\leq 7.5k$ 行的增量功能提交
- **内核树零 submodule**: 23 个 submodule 全部是用户态应用 / 库 / 工具链 (vim、git、LVGL 等), 隔离于 user/external/
- 内核唯一第三方组件: **lwIP 协议栈**, 目录级隔离于 kernel/external/, 不计入自研工作量
- 非衍生自 Linux / xv6 / rCore / FreeRTOS / RT-Thread / ArceOS 等任何现有内核

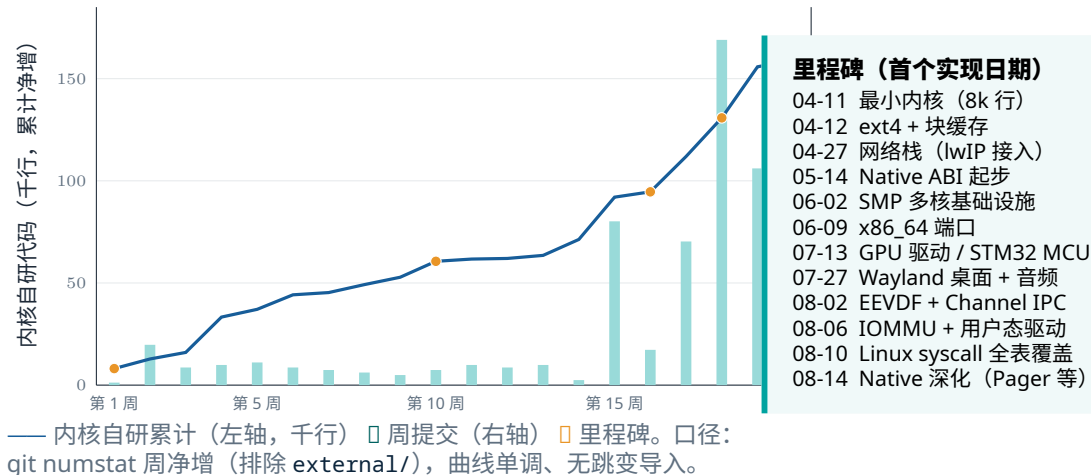
证据即仓库: `git log --reverse` 逐 commit 可查; 第三方边界有编译期门禁审查



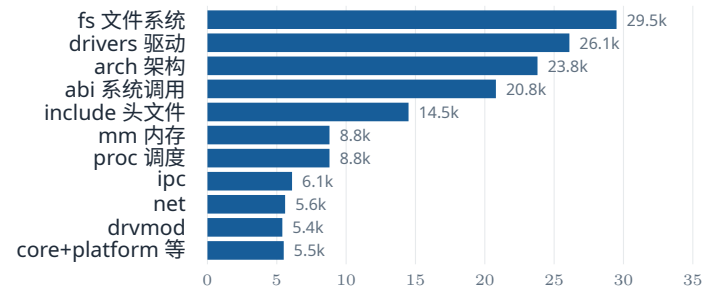
▣ 自研内核 15.7 万 ▣ 自研用户态 3.5 万 ▣ lwIP (隔离) 6.5 万

内核主干 **100% 自研**; 另含 **18 个** 编译器注册驱动、**53 个** 自研用户态命令 / 测试程序。

20 周、483 次提交：从 8k 行最小内核到 16 万行内核



代码分布：19.2 万行自研代码花在哪里



内核自研 15.7 万行（千行 / 模块）；第三方（lwIP / musl）隔离于 external/，不计入。

工作量最高的三块

文件系统 29.5k: VFS + ext4(JBD2) + FAT32 + NTFS 等 6 类后端

驱动 26.1k: virtio 全家桶 + USB/x-HCI + HDA + PCI/ECAM

架构 23.8k: 7 架构 HAL + SMP + 异常/信号上下文

- 统一 Makefile 七架构构建矩阵
- 文档化锁契约与并发门禁
- 可选 ccache 透明加速

跨平台能力矩阵：一套代码，八种指令集

架构	SMP	NOMMU	GUI	真机 / VM
RISC-V 64 *	✓	✓	✓	✓ 星光 2
LoongArch 64 *	✓	-	✓	✓ LS2K1000
aarch64	✓	✓	✓	✓ VirtualBox
x86_64	✓	-	✓	✓ VirtualBox
arm32	-	✓	✓	-
riscv32	-	✓	-	-
ppc64le	-	-	-	-
armv7m (MCU)	-	✓ (仅)	-	✓ STM32

8 个 ISA全部可构建启动；**4 架构** SMP 已验证；**5 架构**可关 MMU 构建 (smoke-arch-mm-matrix 把关)；**5 架构** GUI 行为冒烟。

* = 决赛双架构；make all 产出 kernel-rv/la 与 disk(-la).img；VirtualBox 经 UEFI + ACPI 引导。

02

核心架构

一个内核、两套接口：Linux ABI 负责兼容生态，Native ABI 探索下一代安全内核 API；混合内核按“高频路径留内核、易崩服务迁用户态”判定分层。

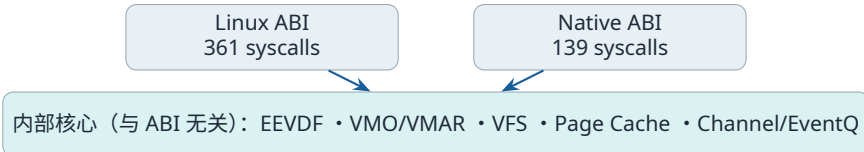
双重 ABI：一个内核、两套接口

Linux ABI 兼容层 kernel/abi/linux/

- **361 个 syscall** 调度表全表覆盖，无空位占位
- 现代机制：io_uring、landlock、pidfd、userfaultfd、mseal、perf_event_open
- 直接运行 musl 生态与完整 Linux 用户态

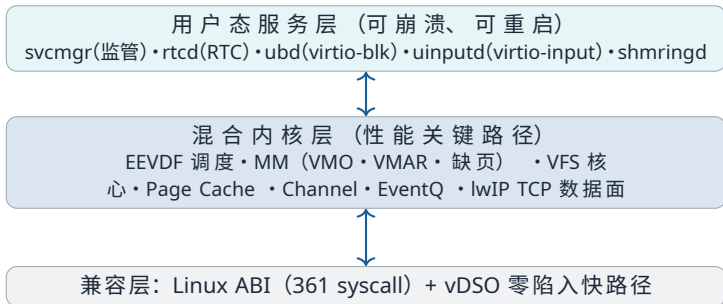
Native ABI 能力层 kernel/abi/native/

- **139 个原生 syscall**，Capability 对象句柄
- VMO / VMAR、Channel、EventQ、Pager
- 无 fork/exec/signal/ioctl 的干净底座



设计价值：既有 Linux 用户态生态（musl、gcc、vim、rustc、FFmpeg），又为下一代安全内核 API 提供演进底座。

混合内核架构形态

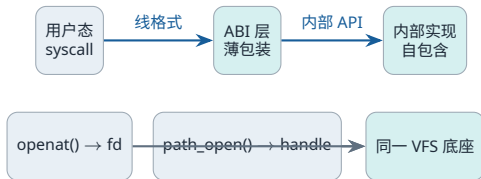


判定规则：**高频 / 延迟敏感路径留在内核**；**崩溃频繁、协议解析类工作迁到用户态服务。**

分层原则：内部实现 ⊗ ABI 薄包装

- 调度、MM/VMA、VFS、页缓存等状态归属**内部核心层**，与 ABI 无关
- ABI 层只做**线格式翻译**，内部 API 单向依赖
- 内部 IPC 头位于 `kernel/include/ipc/`，不包含任何 `abi/` 内容
- Linux 线格式常量由 `kernel/include/core/*.h` 自持

收益：任意新 ABI (含 Linux) 直接包装内部机制即可受益，无需复制实现；门禁 `check-abi-boundary` 编译期审查依赖方向。



锁契约：SMP 并发的基础

```
1  /* 全局锁序契约（节选） */
2  cg_node.lock -> proc_lock -> runq.lock
3                      -> pfa.lock
4  proc_lock -> signal_state.lock
5  proc_lock -> files_struct.lock
6                      -> VFS locks
7  proc_lock -> mm_struct.lock
8  proc_lock -> a20_handle_table.lock
9  g_lwip_lock -> g_net_lock
```

全文：docs/drivers/lock-order.md；新锁必须先文档化再合入。

- 摒弃全局大锁，确立严格**局部锁层级**
- 本地调度 picker 只持本 CPU runqueue 锁（唯一例外）
- ext4 命名空间锁改**读写锁**：8 个并发 rustc 的 dcache 查找互不排队
- 8 核压力门禁常驻：make smoke-sched-stress 等

核心规则：持 runq_lock 时永不取 proc_lock；持 spinlock 时永不阻塞；持设备 / lwIP 锁时不调用 VFS/mm/调度器（除非文档化为非阻塞）。

03

并发与调度

per-CPU EEVDF 调度器：nice 权重比 489:1、基准时间片 10 ms 运行时可调；tokenized park/wake 消除丢失唤醒；8 核压力门禁常驻回归。

EEVDF 调度器：公平与延迟的统一

- **按权记账**：nice 真实控制 CPU 份额（权重表与 Linux 同源）
- **系统虚拟时间**资格门控 + 虚拟截止时间
- RT（级 0，SCHED_FIFO/RR）恒优先于 EEVDF
- **空闲窃取**：本地队列空时拉取远端富余任务
- sleeper bonus 由 EEVDF_MAX_LAG 钳制（10 ms）
- /proc/a20/sched_base_slice 运行时切换桌面 / HPC 形态

关键文件：kernel/proc/sched.c；设计文档：
docs/eevdf-scheduler.md

EEVDF 核心公式

$$vruntime += dt \times \frac{NICE0}{weight}$$

$$vtime += dt \times \frac{NICE0}{\sum weight}$$

$$\text{eligible: } vruntime \leq vtime$$

$$deadline = vruntime + \frac{slice \times NICE0}{weight}$$

nice -20 / +19 权重 43020 / 88，份额比约 **489:1**；
基准 slice 10 ms。

Tokenized Park/Wake 与任务所有权

- 三个**相互独立**的状态维度：语义状态 / 调度所有权 / Park
- (task, wait_seq) token 消除**丢失唤醒窗口**
- on_rq → dispatching → on_cpu 显式所有权交接
- timeout 最小堆：O(log n) 取消，**唯一摘除**
- 唤醒原因显式区分 event / timeout / signal / exit / cancel

关键文件：kernel/proc/park.c、
kernel/proc/sched.c、
kernel/include/core/sync.h



wait_seq 每次 prepare 递增；延迟事件必须同时匹配 task 与 wait_seq，否则视为旧 token 丢弃。

SMP 负载均衡与持久抢占

- **Per-CPU 运行队列**：消除全局调度锁
- 空闲窃取只发生在本地队列空时，**只窃 EEVDF 任务、远端 $nr_running \geq 2$ 、尊重 affinity**
- 跨队列迁移按**CPU 编号升序**锁定，runqueue 引用全程有效
- **持久 need_resched**：远程唤醒先发布状态，再置标志
- IPI 只负责**通知**，不在任意中断上下文直接切换；请求在 syscall / trap / timer 返回等安全点消费
- 远程唤醒统一走 priority-preempt 快路径，**两个 ABI 共享** (pipe / AF_UNIX / futex / channel)

关键文件：kernel/proc/sched.c、kernel/include/proc/proc.h；8 核压力门禁 make smoke-sched-stress，双架构 1 核 / 8 核累计矩阵 make check-proc-step8-local

04

内存管理

VMO/VMAR 解耦物理页与地址空间；大页降级、COW、MAP_SHARED 脏页生命周期完整；Pager 把缺页变成用户态可处理的 channel 消息。

VMO/VMAR：解耦物理页与地址空间

- **VMO** (Virtual Memory Object)：物理页容器，可 resize、共享、按页 fault
- **VMAR** (Virtual Memory Address Region)：地址空间布局，负责映射与权限
- 保护公式： $\text{effective_prot} = \text{vmo_prot} \cap \text{vmar_prot} \cap \text{map_prot}$
- 共享映射：多个 VMAR 映射同一 VMO，实现**零拷贝共享内存**

```
1 // VMO resize / commit / decommit
2 a20_vmo_set_size(h, new_size);
3 a20_vmo_op_range(h, COMMIT, off, len);
4 a20_vmo_op_range(h, DECOMMIT, off, len);
```

```
1 // 零拷贝共享内存 IPC
2 a20_vmo_create(PAGE_SIZE, 0, &vmo);
3 a20_vmar_map(sender, vmo, ...);
4 a20_channel_write(ch, ..., &vmo, 1);
5 a20_vmar_map(receiver, vmo, ...);
```

关键文件：kernel/mm/a20_vmo.c、kernel/abi/native/vmar.c；压力门禁 `make smoke-mm-stress`

大页降级、COW 与 MAP_SHARED 一致性

- `mm_demote_huge_page()`: 将 2 MiB 大页安全拆分为 4 KiB 页
- `mm_fork_clone_present_level()`: fork 时建立**写时复制**映射
- `mm_sync_shared_dirty_for_vnode()`: 维护 MAP_SHARED 脏页生命周期
- **per-mm TLB invalidation transaction**: 旧 frame 延迟到远端 shutdown 完成后才释放 (并行 rustc 崩溃的根因修复)
- `mm->lock` 锁模型保证多核 `mmap/munmap/fault` 并发安全; `mseal` 封禁 VMA

关键文件: `kernel/mm/vm.c`、`kernel/fs/page_cache.c`、`kernel/include/mm/vm.h`

压力门禁: `make smoke-mm-stress`
(write/read、fsync、truncate、fork、remap、eviction)

Pager: 用户态驱动的按需分页

- **PAGED VMO** 缺页不静默填零，而是把页请求作为 channel 消息交给 **pager 线程**
- pager 用 `pager_supply_pages` 回填后唤醒 faulting 线程
- 对齐 Zircon `zx_pager_*`，去掉 `fd / ioctl` 面
- 页供给不跨越 BLP 标签（源 `label ≤ VMO label`）

关键文件：`kernel/mm/vmo.c`（PAGED 分支）、`kernel/abi/native/sys_native_ipc.c`；验证 `make smoke-native-deepen`

页请求消息

类型 `READ/WRITE`（缺页访问类型），`data0` = 缺失页在 VMO 内字节偏移；pager 回填后唤醒该页的 `waiters`。

适用场景

持久化内存、用户态 `swap pager`、虚拟化内存后端、快照 / 迁移。

05

Linux ABI 兼容

361 个 syscall 调度表全表覆盖、无空位；io_uring / landlock / pidfd / mseal 等现代机制已实现；动态链接 musl 程序直接运行。

全量 syscall 覆盖：361 项，无一空位

- **361 个 syscall** 调度入口，四条主线架构编号**全表覆盖**
- x86_64 表扩展至 **463 槽**；LoongArch 私有 file_getattr/setattr
- 每个注册项都有真实 handler——**没有任何固定返回 -ENOSYS 的占位槽**
- 逐项语义等级（full / partial）记录在案，不虚报完成度

覆盖口径：

kernel/abi/linux/syscall_coverage.md；验证
make smoke-abi-linux、make smoke-syscall-ext

已实现的高复杂度机制

- openat2 (RESOLVE_CACHED)、renameat2、statx
- futex PI + robust、membarrier
- splice/tee/vmsplice、eventfd/timerfd/signalfd
- ptrace SEIZE/INTERRUPT、fanotify、keyring、AIO
- mseal VMA 封禁、sched_attr 完整解析

动态链接与用户态生态

- `exec` 正确处理 shebang 与 ELF interpreter (`/lib/ld-musl-*.so.1`)
- `MAP_PRIVATE` / `MAP_FIXED`、TLS (tp 寄存器) 正确建立
- 全部 musl 动态链接的二进制可直接运行
- 用户指针边界显式处理 (identity-mapped 架构下不误判 `argv/envp` 来源)

关键文件: `kernel/proc/exec.c`、
`kernel/fs/vfs/path_resolution.c`

可运行的用户态生态

musl • git • vim • gcc • fastfetch • mksh
• coreutils • rustc (Cargo) • FFmpeg

这是完整桌面能起来的地基；早期 musl 加载期空指针崩溃即从 `exec/mmap` 语义修正而来。

vDSO：零陷入时钟快路径（实测 -44% 延迟）

- `clock_gettime / gettimeofday / getcpu` 用户态零陷入
- riscv64 与 **loongarch64** 双架构实现，经 `AT_SYSINFO_EHDR` 导出
- `vvar` 数据页与内核 `timekeeping` 位级一致，`seqlock` 保护
- `musl` 的 `VDSO_USEFUL` 自动接管，程序零改动获益

关键文件：`kernel/vdso/`；验证 `make smoke-clock-vdso`



LA64 QEMU TCG 实测：-43.7%。

06

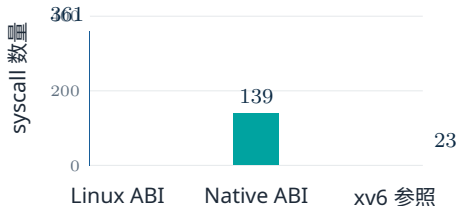
Native ABI

139 个能力化 syscall 表达 361 个 Linux syscall 的功能面（约 2.6 倍压缩）；14 位权利掩码 + BLP 标签 + 时间能力构成句柄安全模型。

Native ABI 设计哲学：2.6 倍接口压缩

- **系统调用压缩**：361 Linux → **139 Native** (约 2.6×)
- **统一抽象**：handle_set_meta 替代 chmod/chown/utimes/truncate；handle_transfer 统一 splice/sendfile/tee 等
- **版本化 ABI**：复杂结构以 size + version 开头，运行时协商
- 无 fork/exec/signal/ioctl；以 task_spawn、Channel、EventQ 替代
- 编号稳定分区 (0x0000-0x0FFF)，遵循 E-APPEND / E-DEPRECATE 演进规则

关键文件：kernel/abi/native/syscall_table.def
(139 项 X-macro 注册)



对照教学内核 xv6-riscv (全表 23 个)：
A20OS Linux 兼容面约为其 **15.7 倍**；Native
以 139 入口表达同一功能面。

Native ABI 深化：把 Linux 机制对象化

- **Pager (0x0D00)**: userfaultfd 的对象化——PAGED VMO + channel 页供给
- **monitor (0x0D10)**: perf 软件事件的对象化——计数对象可订阅 EventQ
- **task_mem_read/write (0x0211/12)**: process_vm 的能力化随机访问
- **vm_share_region (0x030A)**: 按区间导出 VMO 句柄
- **EventQ FS/网络事件源**: vnode-keyed FS 事件 + socket/pipe 就绪边沿

设计文档：

docs/native-abi/09-native-abi-deepening.md;

验证 `make smoke-native-deepen`

三分设计原则

- **应包装**：以更干净的对象接口表达
- **应拒绝**：SysV IPC、io_uring、Landlock 保持 Linux-only，用 capability 替代
- **已有等价物**：pidfd → TASK handle、memfd → MEMORY handle

Handle 生命周期与安全模型

- **14 位权利掩码**: READ / WRITE / EXEC / STAT / DUP / TRANSFER / MAP / WAIT / CONNECT / ACCEPT / CONTROL / ADMIN / SIGNAL
- **权利单调递减**: handle_dup 与 IPC 转移只能缩小权利
- **Bell-LaPadula** 多级安全标签 {L, M, H}: no-read-up / no-write-down
- **Temporal capability**: 到期时间 + 操作次数限制, 自动失效

关键文件: kernel/abi/native/handle_table.c; 设计 docs/native-abi/06-security.md; 验证 make smoke-native-contract

Handle 不变式

同一 Handle 在一个进程表中只存在一份有效引用; IPC 共享传递是原子的, 不存在“悬挂引用”。接收方空间不足时整体返回 NO_SPACE, 杜绝 partial delivery。

07

混合内核服务与 IPC

Channel/EventQ 异步 IPC：单条消息 64 KiB + 8 Handle，要么完整交付要么不交付；svcmgr 崩溃自动重启 + 资源硬隔离；virtio-blk / RTC / input 已整体用户态化。

Channel / EventQ: 异步 IPC 协议

- **两阶段写入**: reserve → 拷贝 → commit; 失败可 abort
- 单条消息最多 **64 KiB 数据 + 8 个 Handle**
- Handle 共享语义: 发送方保留原 handle, 对象引用计数递增
- channel_call (0x0508) **一次陷入完成请求 + 回复**; UP 下时间片捐赠直接切换
- **统一等待对象层**: futex / EventQ / channel / pipe / socket 共用 wait_queue_t

关键文件: kernel/ipc/a20_channel.c、
a20_event.c; 验证 make smoke-native-ipc

SPSC 共享内存环

全相对偏移布局(跨进程可用); Dekker futex 门铃; **非满非空路径零 syscall**; 16 MiB 跨进程完整性验证。

交付保证

消息要么完整交付 (数据 + 所有 Handle), 要么完全不交付。

服务监管者、注册表与健康探针

- **服务监管者** (svcmgr): 声明式清单 + 依赖 + **崩溃自动重启** (指数退避 + flap 预算)
- **服务注册表**: 按名解析 + 崩溃后自动重绑 (CAS claim)
- **健康探针**: ping + 超时强杀, pong 通道随重启重注册
- **资源硬隔离**: 句柄配额 4096 + 对象计数审计 (/proc/a20/objects), 崩溃循环零漂移

验证: smoke-native-svc、smoke-native-registry、smoke-native-isolation、smoke-a20-channel

Linux ABI 透明获益

Linux 程序经
SYS_a20_channel_pair(900) /
SYS_a20_registry_client(901) 以
fd 表面消费同一 channel 机制与服务注册
表——Native 机制, Linux 程序零改动可用。

用户态驱动：能力化设备访问

- **MMIO 白名单授权 + IRQ → EventQ:**
goldfish RTC 整体用户态化 (rtcd)
- **virtio-blk 用户态驱动:** 零拷贝 DMA, FAT32 挂载 (ubd); 页缓存留内核, 块请求经共享环转发
- **驱动崩溃恢复:** 在飞请求失败传导 + 自动重挂载 (ubd_recover)
- **动态设备所有权:** device_claim/release, 任务退出自动释放
- **DMA 缓冲只能来自内核 VMO**
(device_alloc_dma 预物化连续堆)

验证: smoke-native-rtcd、smoke-native-ubd、
smoke-native-shmring、smoke-iommu-discovery

IOMMU 硬件隔离

riscv-iommu-pci bring-up: DDT/CQ/FQ 编程使能, devid 0 SV39 静态翻译域, TR_REQ 验证已映射 / 未映射 IOVA。

设备身份

*.a20drv 描述段声明 placement / 设备类型 / 稳定名 / 匹配表; 一个设备一个 owner。

KEP: 内核扩展点 (安全版 eBPF)

- 与 Linux LKM 本质不同: **受限字节码程序**, 非任意原生代码
- 标准 eBPF 指令编码 (struct bpf_insn), **同一份字节码**经 Native (ext_prog_load) 或 Linux (bpf(2)) 加载
- 无指针、无任意内存访问、无循环 (严格向前跳转, **结构性终止**)
- 线性扫描**验证器**逐条检查, 非法即拒绝
- 扩展点 = 类型化上下文; 进程退出自动清理

关键文件: kernel/drvmod/
kernel/abi/native/sys_native_ext.c、
docs/extensions.md

第一个扩展点: syscall-filter

上下文 8 字: nr, arg0..5, abi。附着于每次系统调用入口, R0 判定放行 / 拒绝 / 终止。

验证

```
make smoke-native-ext、make bpf_smoke
```

08

文件系统与网络

高兼容 VFS: openat2 / renameat2 语义完整, ext4 带 JBD2 日志恢复; lwIP 事件驱动集成, 无编译期网络默认值。

高兼容 VFS 与路径解析

- `openat2` 语义: `RESOLVE_NO_XDEV`、`BENEATH`、`NO_SYMLINKS`、`RESOLVE_CACHED`
- `renameat2`: `RENAME_EXCHANGE`、`RENAME_NOREPLACE`、跨 mount 拒绝
- 正确处理 `mount root`、`..` 穿越、符号链接深度限制 (`MAX_SYMLINKS=40`)
- Page Cache 与 block cache 统一, 保证多核一致性; 并发写回按 dirty generation 定序
- dcache 目录项缓存 + 模块化 `vfs/*.c` 单元

关键文件: `kernel/fs/vfs/path_resolution.c`、
`dcache.c`、`mount_ops.c`、`stat_perm.c`

关键不变式: `lookup` 必须持有父目录引用; `rename` 原子更新目录项与 inode `nlink`; 跨后端返回 `-EXDEV`。

文件系统后端契约矩阵

后端	rename	link	symlink	关键能力 / 限制
FAT32	×	×	×	元数据仅存 RAM
ext4	✓	✓	快链 ≤ 60B	JBD2 journal 恢复, 64 位文件索引无 B-tree 分裂
NTFS	✓	×	×	
ISO9660	×	×	×	
ramfs	✓	✓	✓	只读 CD-ROM
pipe	-	-	-	max inodes 4096
				PIPE_BUF 原子写

伪文件系统: devfs (动态 class 设备)、procfs (快照文件)、sysfs (/sys/class/{char,block,net,input,display,audio})、cgroupfs。
契约文档: docs/fs/fs-consistency-model.md、docs/fs/vfs-edge-semantics.md

事件驱动网络栈 + 运行时配置

- lwIP 以 NO_SYS=1 集成，避免多线程轮询
- kernel/core/progress.c 统一 **bottom-half** 进展机制
- virtio-net RX/TX 由**中断 + 事件**驱动
- 调度器切任务前运行 bottom-half，降低上下文切换开销

kernel/net/lwip_stack.c、
kernel/core/progress.c、
kernel/drivers/net/virtio_net.c

- **无编译期网络默认值**，缺失配置返回 -ENETUNREACH
- 内核命令行 a20.ip/gateway/dns/dhcp + /proc/net/config 运行时展示
- 套件：TCP/UDP/ICMP、DNS、AF_UNIX、AF_ALG、SCM_CREDENTIALS、netlink uevent

调用路径：网卡中断 → a20_lwip_poll() → progress_run() → socket bottom-half → 唤醒等待任务

09

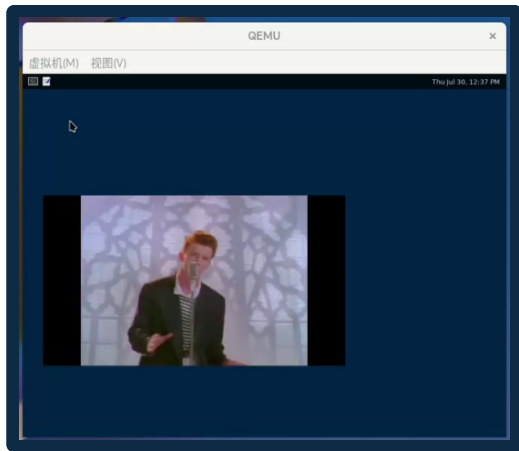
图形与桌面

内核原生 DRM/KMS + evdev 支撑 Weston / labwc Wayland 桌面与 LVGL 自研桌面；virtio-gpu virgl 打通 3D 命令透传链路。

Wayland 桌面：内核原生支持（QEMU 实拍）

- 从源码**移植** wlroots / labwc / weston / xfce 4.20 组件，submodule 保持纯净
- **内核补齐**而非改组件绕行：少量用户态补丁构建期临时应用并还原
- 原生 libinput 后端经 libseat 打开 /dev/input/event*
- 帧调度链完整：commit → render → pageflip → vblank
- **LVGL 自研桌面**：仪表盘 / 终端 / 文件 / 进程监控

关键文件：kernel/drivers/gpu/drm.c；验证 make smoke-qemu-gui-riscv64（QMP 注入键鼠 + 截图判帧）



实拍：riscv64 QEMU，Weston 桌面内播放视频。

内核 DRM 强化与 3D 图形加速

内核 DRM 强化

- **唯一 KMS 对象 ID**: plane / crtc / connector / encoder 互不冲突
- 页翻转事件: FIFO 不覆盖、seq 单调、时间戳单调钟、EBUSY 语义
- 标准 32 字节 UAPI 结构, 杜绝栈溢出
- EDID blob property (含合成合规 EDID)

3D 链路: 用户态 → ioctl(/dev/dri/card0) → drm_ioctl → gpu_ioctl → SUBMIT_3D → virglrenderer → 宿主 GPU; 自测工具 gpu3d_test

3D: virtio-gpu virgl

- **内核 3D 命令路径完整**: feature 协商、capset、context / resource / submit 透传
- 渲染在宿主机, guest 内核只搬运命令与资源
- A20_GPU_IOCTL_* 经 /dev/dri/card0 暴露
- 与 2D fbdev 路径互不干扰, 无 virgl 自动回退

10

性能

面向 BuildStorm 热区的 14 项优化全部保留真实计时；双架构 BuildStorm 2551/2315 s → 1483/1399 s，用例 180/180 全过。

性能优化体系（面向 BuildStorm 热区）

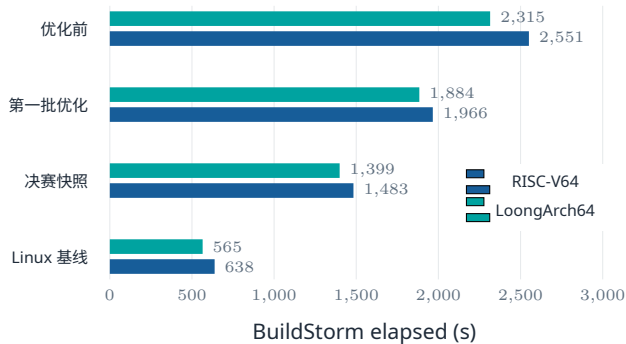
- **用户 vDSO**: riscv64 / loongarch64 clock_gettime 零陷阱
- **COW 页错误 TLB 定向失效**: 广播 SBI/IPI → 只向 mm->active_cpus 定向
- **批量延迟 frame 释放**: frame_put_many 每 64 帧只取一次 pfa.lock
- **ext4 命名空间读写锁**: 8 个 rustc 的 dcache 查找不再互斥
- **VMA 索引 256 → 1024**: 编译器进程保持 $O(\log n)$ 二分
- **PRIVATE futex 跳过页表遍历**; fault 去冗余检查
- per-vnode 共享映射开关 + 热路径索引化, 消除全局脏页扫描

方法论

每项保留真实计时; 数字只按干净提交固定, 不外推。

文档: docs/BuildStorm-2026.md;
mm/futex/sched 等全组压力验证

决赛 BuildStorm: 双架构 180/180, 耗时下降约 40%



快照 d5ae5a16 / f3e3ce48 / f9732348, 同一 workload、8 vCPU、TCG multi; 与 Linux 基线跨 QEMU 版本, 按各自证据边界解释。

-41.9% / -39.6%

RV / LA 相对优化前耗时下降

180/180

BuildStorm 用例通过 (双架构)

199.10/200

CAgent 得分 (10/10 通过)

决赛快照 f9732348: exit 0 主动关机, 产物 SHA-256 存档。

11

驱动与平台

混合驱动模型：零开销 MMIO + 统一 hwapi + 双部署 profile；drvmod 可加载模块约 80 个导出符号
唯一 API 面；下至 64 KiB SRAM 的 MCU。

混合驱动模型

- **零开销 MMIO**: 板级地址用宏内联, readl/writel 编译为单条 load/store
- **统一 hwapi**: request_irq、dma_alloc、clock_get_cycles 跨架构抽象
- **类 ops vtable**: block_dev_ops_t、net_dev_ops_t、char_dev_ops_t
- 编译期注册: DRIVER_REGISTER / BOARD_REGISTER 放入 .driver_init linker section
- 双部署 profile: generic (DriverStore + .a20drv) / embedded (静态链接, 决赛默认)
- drvmod: ET_REL 装载 + .a20drv 描述段 + veneer/GOT, **约 80 个导出符号**唯一 API 面

关键文件: docs/drivers/README.md、kernel/include/drivers/hwapi.h; 验证 make smoke-drvmod

音频子系统

virtio-snd • Intel HDA (PCI class 驱动, CODEC 拓扑发现) • PC Speaker; /dev/audioN + capability/format 校验; audioplay 与 FFmpeg 播放器

USB 协议栈

host / class / core 分层; xHCI; HID / 存储类; make smoke-usb-x86_64

PCI 与平台设备

ECAM 枚举 • **e1000 真 PCI 网卡** • ahci • virtio-scsi • 平台网卡 (starfive_gmac / ls2k_gmac) • dw_sdio

VirtualBox

x86_64 / aarch64 经 **UEFI + ACPI + ECAM** 引导; VMSVGA / E1000 / VirtIO-SCSI

嵌入式与 MCU：同一内核，64 KiB SRAM 起步

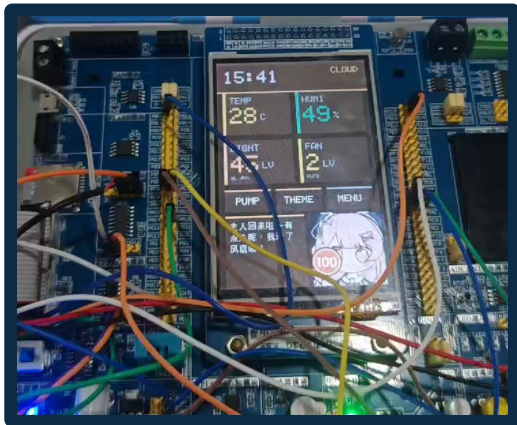
STM32F103 (ARMv7-M / NOMMU)

- 64 KiB Flash / 20 KiB SRAM bring-up (QEMU 验证)
- 普中玄武 ZET6: 512 KiB Flash / 64 KiB SRAM
- 外设: LCD、舵机 / 执行器、DHT11 传感器、蓝牙、ESP8266、背光

PPC64LE 与跨平台工程

QEMU pSeries 固件 (MMU 单核边界); 统一 arch/HAL 抽象 + 七架构构建矩阵, 物理板与虚拟平台同构验证。

入口: `make stm32f103-xuanwu`、`make run-stm32f103-qemu`、`make ARCH=ppc64le run`



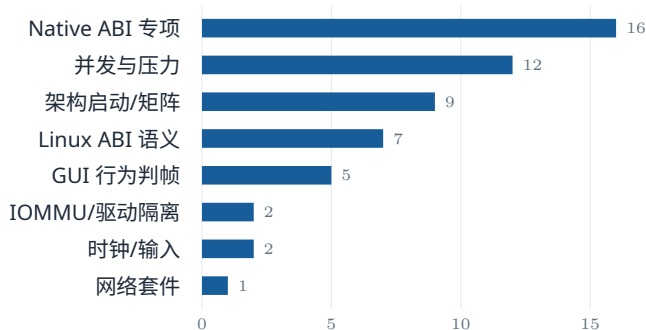
实拍：普中玄武 STM32F103ZET6 运行 A20OS，LCD 仪表盘 + 舵机 / DHT11 / 蓝牙外设。

12

工程化与验证

54 项行为冒烟门禁 + 15 项静态/运行时审查门禁；文档即事实契约保证设计与实现不漂移；/proc/a20 暴露运行时对象与调度计数。

测试门禁体系：54 项冒烟全部可复现



54 项 `make smoke-*` 按类别分布；明细见备查 D。

54

`make smoke-*` 行为冒烟门禁

61

项 `check-*` 审查 / 构建门禁

30+

自研压力 / 语义测试工具 (`user/cmds`)

调度压力矩阵 `check-proc-step8-local`：
双架构 1 核 / 8 核累计。

- DOCS_AS_FACT_CONTRACT：架构文档描述**当前实现**，未来设计放规划材料
- 20+ 静态审查门禁：锁模型、ABI 边界、I/O 进展模型、文档漂移等
- 外部依赖边界审查（musl / lwIP / 用户态工具）
- Docker 交叉编译环境 + 可选 ccache
- 92 篇设计文档、1.6 万行，随代码同仓演进

崩溃 / 重启循环后对象计数回归基线，泄漏审计可重复验证。

/proc/a20/ 运行时状态面

- objects：7 项实时对象计数 + 句柄硬配额审计
- task_lifetime：task/ref、runqueue、wait/wake、timeout、migration、IPI、violation 计数
- sched_base_slice：EEVDF 虚拟 slice 旋钮

13

未来工作

下一步工作

1. **I/O 完全事件驱动**: 收敛块 / 网络设备最后的轮询 fallback 路径
2. **网络栈深化**: 降低 g_lwip_lock 竞争, 扩展 socket 并发测试
3. **VFS / Linux ABI 语义收紧**: partial → full 逐组提升
4. **IOMMU 动态 DMA**: per-device domain 与 fault 队列消费, 端到端硬件隔离
5. **Native 多架构矩阵**: 将 16 项原生测试扩展至全架构 CI-like 运行

核心追求: 在保证 Linux 生态兼容的前提下, 构建更安全、更高性能、更可维护的下一代内核底座。



Q & A

感谢各位专家评委，恳请批评指正

感谢全国大学生计算机系统能力大赛平台 · 感谢开源社区 AOSC、plos-clan 的支持

gitlab.eduxiji.net/T202610486999803/oskernel2025-a20 | github.com/fQwQf/A200S

备查 A: EEVDF 正确性与参数

- 权重表 `sched_prio_to_weight[40]` 与 Linux 同源：
`nice -20 → 43020`, `nice +19 → 88`
- `EEVDF_NICE0_LOAD = 1024`, 基准 slice 10 ms,
`EEVDF_MAX_LAG` 10 ms
- `eligibility`: $vruntime \leq vtime$; `pick`: `eligible` 中最小 deadline
- 延迟唤醒补偿按 lag 计入 `vruntime`, 但钳制到 `MAX_LAG`, 防止“睡一次吃光 CPU”

可能被问：与 Linux EEVDF 的差异

数据结构为 per-CPU 队列 + 显式所有权状态机，非 `rbtree` 全局队列；RT 单独级 0 恒优先；slice 经 `/proc/a20/sched_base_slice` 运行时可调，用于桌面 / HPC 形态切换。

证据：`kernel/proc/sched.c`、`kernel/proc/proc_internal.h`、`docs/eevdf-scheduler.md`

备查 B：并行编译崩溃的根因与修复

- **现象**：8 核 / 8 job 全量 cargo 编译下，RISC-V64 rustc ICE；LoongArch64 SIGILL / SIGSEGV / 地址异常
- **根因**：清除或替换 PTE 后，旧 frame / page-cache 引用可能在**远端 TLB shutdown 完成前**被释放并复用——新写入落到旧物理页
- **修复**：per-mm TLB invalidation transaction，旧 backing 延迟到 shutdown 确认后释放；LoongArch64 同步等待远端 shutdown；统一 VMA deferred flush、slab、block-cache、buddy 的并发边界
- **另两处持久化损坏**：VirtIO packed used ring 16 位 idx 撕裂读（固定布局 + barrier）；block-cache 并发写回旧 ext4 bitmap 晚于新快照落盘（writeback/fill 锁 + dirty generation 定序）

完整分析与证据边界：docs/BuildStorm-2026.md §2；验证：并发 rustc + 官方测试组 + 定点 probe 交叉验证

备查 C: 361 / 139 的统计口径

- Linux ABI: `syscall_table.def` 注册 **361 个调度项** (含 2 个 A20 扩展、5 个 x86_64 legacy 槽), 每个注册项都有真实 handler
- 少数 `-ENOSYS` 返回是 Linux 本身的架构 / 版本正确语义 (如 `nfsservctl` 已于 Linux 4.19 移除)
- 语义等级逐区标注 full / partial, 见 `syscall_coverage.md`, 不夸大兼容度
- Native ABI: `syscall_table.def` 中 139 条 X-macro 注册

验证方法

`make smoke-abi-linux` 逐组语义验证; `smoke-syscall-ext` 覆盖 `io_uring` / `landlock` / `userfaultfd` / `perf` 等现代机制; 真实负载: musl 生态 + Cargo 全量编译。

备查 D: 54 项冒烟门禁明细

数量	类别	目标
16	Native ABI 专项	smoke-native-{contract,deepen,ext,futex,ipc, isolation,linux,mm,registry,rtcd,shmring,svc,ubd,...}
12	并发与压力	{mm,sched,futex,vfs,proc,procfs,pty,scm,signalfd,evdev}-stress 等
9	架构启动 / 矩阵	smoke-<arch> × 7 + smp-bringup + arch-mm-matrix
7	Linux ABI 语义	abi-linux、syscall-ext、ptrace、proc-a20、vfs-edge 等
5	GUI 行为判帧	smoke-qemu-gui-{riscv64,loongarch64,aarch64,x86_64,arm32}
2	IOMMU / 驱动隔离	iommu-discovery、iommu-udriver-isolation
2	时钟 / 输入	clock-vdso、dual-input
1	网络套件	smoke-network-suite (TCP/UDP/ICMP/DNS/AF_UNIX)

全部经 `make smoke-*` 一键复现；超时无 PASS 标记即失败（行为判据，非仅编译通过）。